



وزارة الاتصالات
وتكنولوجيا المعلومات



BuggedCart: Web Application Security Assessment & Secure SDLC Integration

A complete workflow:
Build vulnerable app → Assess → Exploit → Fix → Re-test

DEPI Final Project
Course: Information Security Analyst
Instructor: Eng. Amgad Soliman

Team Members:

Yahia Samir
Belal ElDakkak
Ahmed Mohamed
Mohamed Khalil

Contents

1	Executive Summary	2
2	Project Objectives and Scope	2
2.1	Objectives	2
2.2	Ethics and Safety Notes	2
3	Technology Stack	2
3.1	Application Stack	2
3.2	Security Tooling	4
4	System Architecture	4
4.1	High-Level Architecture	4
5	Threat Modeling	4
5.1	Asset Classification	4
5.2	STRIDE Mapping	4
6	Attack Surface Analysis	5
7	Assessment Methodology (Secure SDLC Workflow)	5
8	Vulnerability Scanning Results (OWASP ZAP)	6
8.1	Risk Matrix (Summary)	6
9	Manual Verification and Exploitation Highlights	7
9.1	OWASP A1 SQL Injection Example	7
9.2	OWASP A9 Using Components with Known Vulnerabilities Example	8
10	Secure Coding and Mitigations Implemented	9
10.1	Example: Secure SQL Query Pattern	9
11	Validation and Re-Testing	9
12	Key Outcomes	9
13	Lessons Learned	9

1 Executive Summary

BuggedCart is a purposely vulnerable full-stack web application created to demonstrate a realistic cybersecurity workflow: **designing a vulnerable system, mapping threats, scanning and validating findings, manual exploitation, implementing secure patches, and re-testing** under a Secure SDLC model. The project intentionally included OWASP Top 10-style weaknesses and validated them using automated and manual techniques (OWASP ZAP as the primary scanner, plus browser-based payload verification).

2 Project Objectives and Scope

2.1 Objectives

- Build a custom vulnerable web application (BuggedCart) for learning and testing.
- Perform threat modeling and attack surface analysis before testing.
- Conduct dynamic analysis with OWASP ZAP and confirm results manually.
- Demonstrate exploitability with safe, controlled payloads.
- Patch vulnerabilities using secure coding best practices and re-test.

2.2 Ethics and Safety Notes

This application is intentionally vulnerable and must be deployed only in isolated test environments. Exploitation steps shown in this report are for educational and defensive security purposes.

3 Technology Stack

3.1 Application Stack

- Backend: Flask (Python)
- Frontend: Custom HTML/CSS/JS (hacker-themed UI)
- Database: SQLite (local) and MongoDB Atlas mentioned for alternative deployment paths
- Hosting/Deployment: Cloud deployment options (e.g., Render/Railway/Cloudflare workflows referenced by the team)

These choices align with the project goal of building a realistic but controlled web app attack surface.



Figure 1: Home Page



Figure 2: Used OWASP Top 10

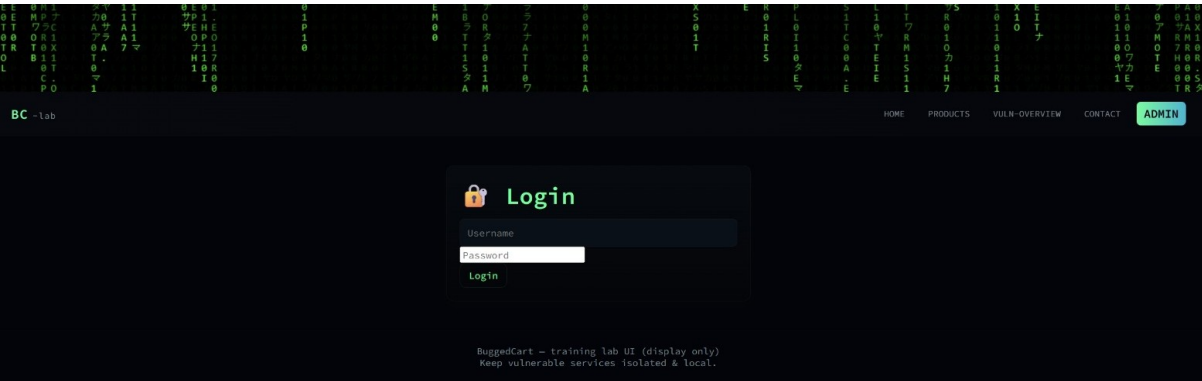


Figure 3: Admin Login Page

3.2 Security Tooling

- OWASP ZAP (primary scanner)
- Manual verification via browser payloads (and optionally Kali tooling as part of the learning environment)

The workflow intentionally combines automated scanning evidence with manual confirmation.

4 System Architecture

4.1 High-Level Architecture

BuggedCart follows a typical web architecture:

- **Client:** user browser sends GET/POST requests
- **Flask Backend:** routes containing vulnerable and fixed variants
- **Database Layer:** SQLite storing users/products/reviews
- **Attacker Model:** malicious payloads via URL parameters, headers, and form inputs

5 Threat Modeling

5.1 Asset Classification

Threat modeling started by identifying critical assets such as credentials, user input channels, backend logic, and cloud infrastructure.

5.2 STRIDE Mapping

Key threats were mapped using STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).

Asset	Type	Importance	Description
Credentials	Confidential	High	Admin & user passwords
Product Data	Internal	Medium	Modifiable via SQLi
User Input	External	High	Main source of vulnerabilities
Backend Logic	Operational	High	Processes all user requests
Cloud Infrastructure	Critical	High	Can be taken over if exploited

STRIDE Category	Description	Example in BuggedCart
S – Spoofing	Impersonating another user	Broken authentication in login page
T – Tampering	Modifying data or requests	SQL injection modifies database
R – Repudiation	Denying actions due to missing logs	No request logging
I – Information Disclosure	Unintended data exposure	Error messages display SQL syntax
D – Denial of Service	Crashing the app or service	Command injection causes freeze
E – Elevation of Privilege	Gaining admin rights	IDOR + SQLi enables privilege escalation

Figure 4: Threat Modeling Tables (Asset + STRIDE)

6 Attack Surface Analysis

Primary entry points and their intended weaknesses include:

- Login page — SQL injection via unsafe query building
- Product search — injection patterns and weak access checks
- Product reviews — stored XSS risk
- Admin contact — OS command injection behavior
- Admin delete — missing CSRF defenses
- URL parameters — IDOR-style access control weaknesses

These were explicitly identified as the app's teaching attack surfaces.

Attack Surface	Vulnerability	Explanation
Login page	SQL Injection	Direct string concatenation without sanitization
Product search	Injection + IDOR	Input directly appended to SQL query
Product reviews	Stored XSS	Review comments rendered unsanitized
Admin contact	Command Injection	<code>os.popen()</code> used directly
Delete button	CSRF	No token or protection
URL Parameters	IDOR	No user authorization checks

Figure 5: Attack Surface Summary

7 Assessment Methodology (Secure SDLC Workflow)

The project follows an SDLC-aligned security workflow:

1. **Plan & Model:** define assets, threat model, and attack surfaces
2. **Build (Vulnerable by Design):** implement OWASP-style weaknesses for learning
3. **Scan:** run OWASP ZAP automated scanning and record alerts
4. **Verify:** reproduce key findings with manual payload tests
5. **Fix:** patch issues using secure coding patterns
6. **Re-test:** validate mitigations through re-scan

8 Vulnerability Scanning Results (OWASP ZAP)

OWASP ZAP produced an initial set of findings (reported as **20 alerts**) including high-risk issues such as injection, missing anti-CSRF tokens, and misconfiguration indicators.

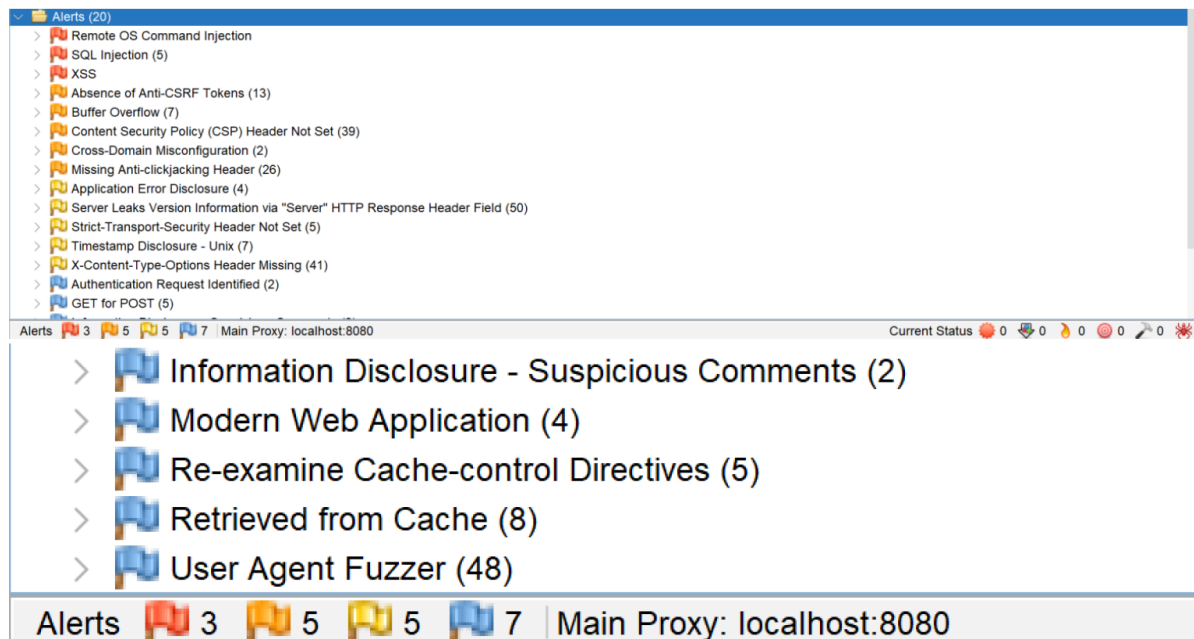


Figure 6: OWASP ZAP Alerts Summary

8.1 Risk Matrix (Summary)

The project used a likelihood/impact approach where SQL injection, XSS, and OS command injection were categorized as high risk.

Vulnerability	Likelihood	Impact	Risk Level
SQL Injection	Very High	Critical	● High
Remote OS Command	High	Critical	● High
CSRF	Medium	Medium	● Medium
Information Disclosure	Medium	Low	● Low
Anri clickjacking header	Medium	Medium	● Medium
XSS	Very High	Critical	● High

Figure 7: Risk Matrix Evidence

9 Manual Verification and Exploitation Highlights

This section documents **controlled demonstrations** showing exploitability.

9.1 OWASP A1 SQL Injection Example

A representative vulnerable pattern was direct string concatenation in SQL queries. The documentation provides an example and a secure parameterized alternative.



Figure 8: SQL Injection Patched Preview

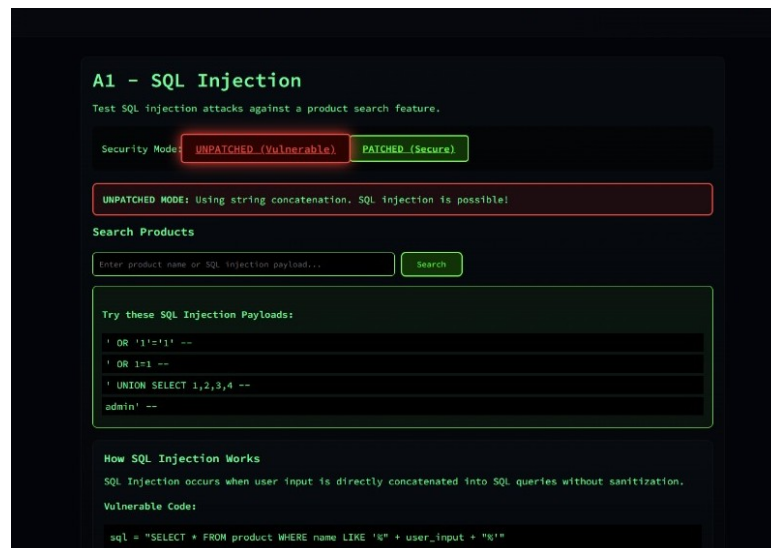


Figure 9: SQL Injection Unpatched Preview

9.2 OWASP A9 Using Components with Known Vulnerabilities Example

This example demonstrates the security risks associated with using outdated third-party components that contain publicly disclosed vulnerabilities (CVEs).

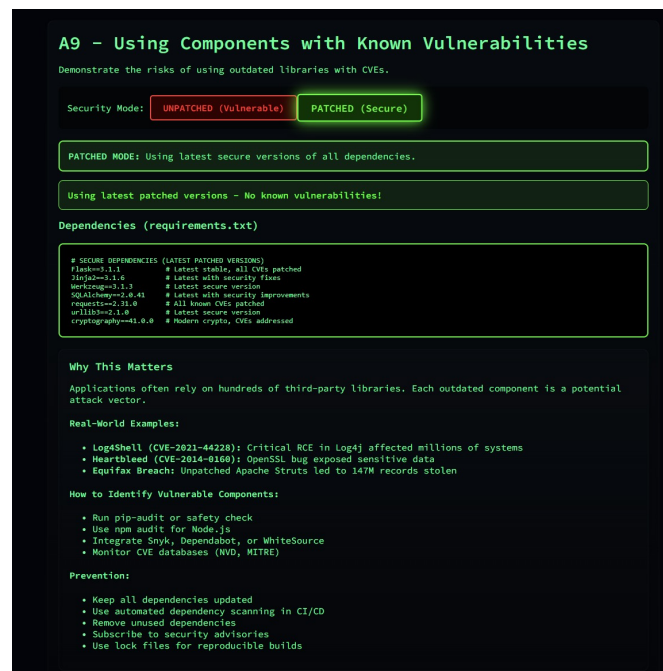


Figure 10: Using Components with Known Vulnerabilities Patched Preview

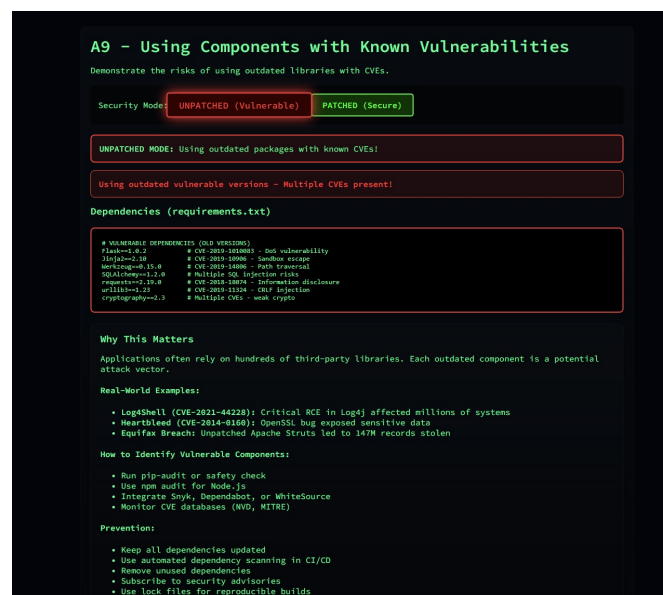


Figure 11: Using Components with Known Vulnerabilities Unpatched Preview

10 Secure Coding and Mitigations Implemented

After confirming findings, the project applied secure coding best practices:

- Replace unsafe SQL concatenation with **parameterized queries**
- Strengthen authentication (hash passwords instead of plaintext)
- Escape user-generated content to mitigate XSS
- Improve access control checks and add CSRF defenses (where applicable)
- Reduce information disclosure and improve security headers

These mitigation directions are explicitly described in the project documentation and presentation.

10.1 Example: Secure SQL Query Pattern

Listing 1: Secure SQL query pattern (parameterized query)

```
1 sql = "SELECT id, name, price, description FROM product WHERE name  
    LIKE ?"  
2 rows = cur.execute(sql, ('%' + q + '%',)).fetchall()
```

11 Validation and Re-Testing

A final OWASP ZAP scan was used to confirm mitigations. The documentation states that after patching unsafe SQL building, improving authentication, escaping output, and strengthening controls, a re-scan confirmed that most issues were mitigated.

12 Key Outcomes

- Built a realistic vulnerable web app training lab (BuggedCart) aligned with OWASP themes.
- Identified vulnerabilities via OWASP ZAP and verified them manually.
- Documented risk and attack surfaces using STRIDE + asset classification.
- Implemented secure coding improvements and validated mitigation through re-testing.

The project demonstrates that even a single open-source scanner (OWASP ZAP) can provide actionable findings when combined with manual verification and a Secure SDLC mindset.

13 Lessons Learned

- Secure SDLC works best when threat modeling happens **before** testing.
- Automated scanners are powerful, but manual verification is necessary to confirm exploitability.
- Most critical web vulnerabilities repeat the same root causes: untrusted input + unsafe execution + weak controls.
- Patching is not complete without re-testing and documenting evidence.

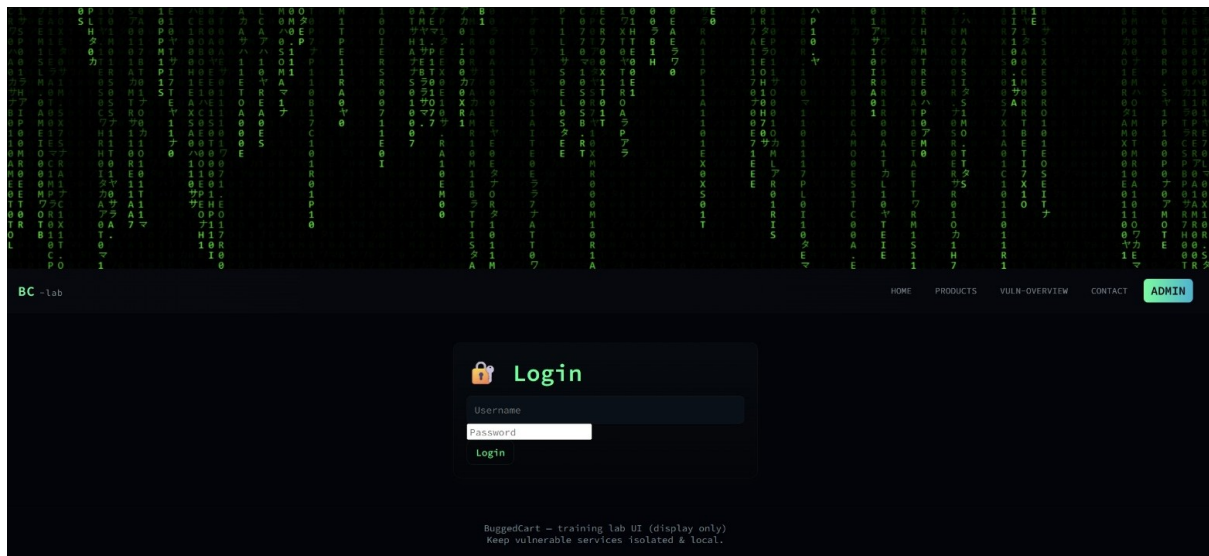


Figure 12: Admin Login Page